158.337 Database Development
Week 2 - The Relational Database Model

Read Chapter 3

The relational database model (RDM) has been successfully implemented in many industries. However, earlier it seemed the Object Oriented database model (OODM) would replace the RDM, given OODM can increasingly manage complex data (video, audio) … yet the OODM fell short in the database arena. However, the OODM's basic concepts have become the basis of a wide variety of database systems analysis and design procedures. In addition, the basic OO approach has been adopted by many application generators and other development tools.

The OO and relational battle for dominance in the database marketplace seems remarkably similar to the one waged by the hierarchical and network models against the relational model almost three decades ago. Both are useful and have their applications. The relational model is primarily geared to business applications, while the OO model tends to focus on very specialized engineering and scientific applications. In the database arena, the most likely scenario appears to be an ever-increasing merging of OO and relational data model concepts and procedures.

Relational Databases

The database is a structure that houses one or more tables and metadata. The metadata are data about data. Metadata include the data (attribute) characteristics and the relationships between the entity sets.

Entity integrity and referential integrity are important database rules because they are the basis for expressing and implementing relationships in the entity relationship model. Entity integrity ensures that each row is uniquely identified by the primary key. Therefore, entity integrity means that a proper search for an ***existing*** tuple (row) will always be successful. (And the failure to find a match on a row search will always mean that the row for which the search is conducted does not exist in that table.) Referential integrity means that, if the foreign key contains a value, that value refers to an existing valid tuple (row) in another relation. Therefore, referential integrity ensures that it will be impossible to assign a non-existing foreign key value to a table.

Data Dictionary - Just as the database stores data that is of interest to the users (e.g., entities), the data dictionary stores data structures that is of interest to the database designer (e.g., the number of tables created, the names of the tables and of each table's attributes, data type of each attribute, relationships between tables, enforced domains, constraints, etc).

How you will name table attributes is also crucial, else it will make the RDBMS's data dictionary cumbersome to use. So avoid homonyms and synonyms.

- Homonyms are when CUSTOMER table has an attribute NAME, and the SALES table also has an attribute NAME. Now, it is unclear ---- Which entity does the attribute NAME belong to?
- Synonyms exist when the same attribute has more than one name.
  Using AGENT_CODE as the foreign key in the CUSTOMER table to reference the primary key AGENT_CODE in the AGENT table is much clearer than naming the CUSTOMER table's foreign key CUST_AGTCODE (or anything else).

Synonyms make it difficult to keep track of foreign keys if they are named differently from the primary keys they point to.

```
CREATE TABLE AGENT (
                        AGENT_CODE VARCHAR2(8) PRIMARY KEY,
                        AGENT_PHONE VARCHAR2(50));

CREATE TABLE CUSTOMER (
                        CUS_CODE VARCHAR2(50),
                        CUS_LNAME VARCHAR2(50),
                        CUS_ZIP VARCHAR2(10),
                        AGENT_CODE VARCHAR2(8) REFERENCES AGENT(AGENT_CODE));
```

Table name: CUSTOMER

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CODE
1132445	Walker	32145	231
1217782	Adares	32145	125
1312243	Rakowski	34129	167
1321242	Rodriguez	37134	125
1542311	Smithson	37134	421
1657399	Vanloo	32145	231

Table name: AGENT

AGENT_CODE	AGENT_PHONE
125	6152439887
167	6153426778
231	6152431124
333	9041234445

Codd's 12 Relational Database Rules

In 1985, Codd published a list of 12 rules to define a relational database system. These 12 rules are based upon set oriented systems, i.e., the DBMS should treat the database as a SET of tuples /rows, and not treat each row separately. Later, a Rule Zero was added … Why the rule is named *zero*?... to maintain these 12 rules rather than create a 13th rule ;-). The zero rule states that the relational facilities must be ingrained in the RDBMS, and not be managed by some external support tool.

Relational algebra concepts

Binary Operators – applied on two relations

The *Union*, *Intersect*, and *Difference/Minus* operator can be applied to a pair of tables which are union-compatible. Note: In order for two relations to be union-compatible, both must have (1) the same number of attributes (columns) and (2) corresponding attributes (columns) must have the same domain.

The *Product*, *Join*, and *Divide* operators can be applied to a pair of tables that are not union-compatible. Divide does place specific requirements on the tables to be operated on; however, those requirements do not include union-compatibility.

Unary Operators – applied on one relation

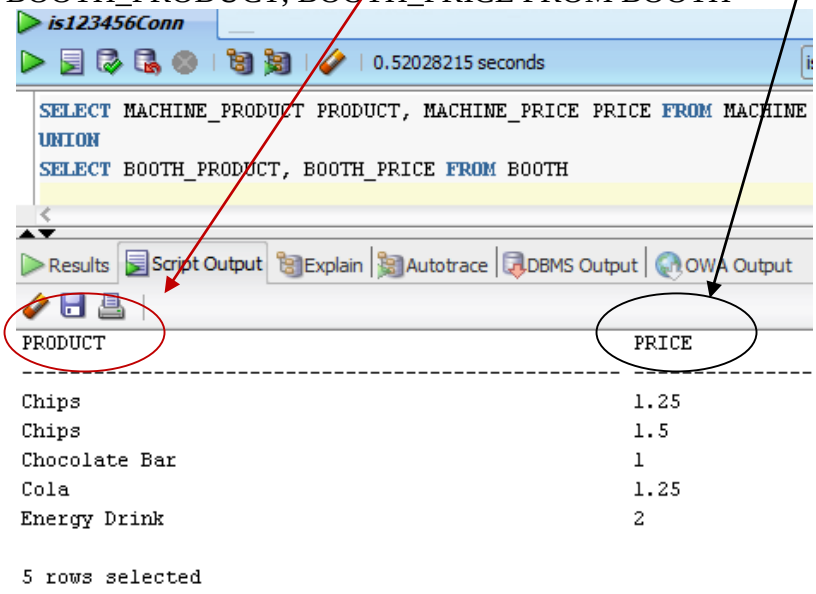
Select (or *Restrict*) and *Project* are performed on individual tables, not pairs of tables. (Note that if two tables are joined, then the result is a single table and the Select or Project operator is performed on that single table.)

Appendix Slides

The Appendix slides given for this week demonstrate use of SQL statements for different algebra operations. This is an extension of the chapter 3 (which introduces you to the concept of set theory and simple SQL language). You need to keep applying these concepts as you experiment with databases; hence you are being introduced to the implementation aspects also.

Try answering questions (slide 15) on your own first. Check your answers after creating tables from given script files (slide 16). You will need to write simple SQL statements if you want to check your answers through SQLDeveloper.

> For example: MACHINE UNION BOOTH will involve the SQL statement, using aliases PRODUCT and PRICE. The aliases are shown circled for your ease in identifying them.
>
> SELECT MACHINE_PRODUCT PRODUCT, MACHINE_PRICE PRICE FROM MACHINE
> UNION
> SELECT BOOTH_PRODUCT, BOOTH_PRICE FROM BOOTH



Note the use of aliases (PRODUCT and PRICE) above.
These concepts will be covered in more detail later (when you start writing SQL statements yourself) – but good to get a grip of these things sooner than later. So…. rather than confuse the reader as to whether the column name should be MACHINE_PRODUCT or BOOTH_PRODUCT, we use an alias and call its imply PRODUCT. Same has been done for PRICE.

You have been given a small tutorial exercise which applies simplistic use of joins. Read the Appendix and analyze these concepts in the given problem domain (tutorial exercise). This will enable basic understanding on usage of relational concepts. Relational concepts have evolved from relational algebra, i.e. predicate logic and set theory. This means we can *logically* make assertions on a group of things which are all members of a *set*, and these assertions can be verified using databases as True/False.

Together the predicate logic and set theory form the basis of performing operations on relations, and lead to the basics of manipulating operations in SQL (which we will be using on an ongoing basis with each chapter reading)